

TP 3 – Unir & Trouver

I Implémentation par arbre en OCaml

Exercice 1. Implémentation par arbre Cet exercice est à réaliser dans un fichier `unirtrouver.ml`.

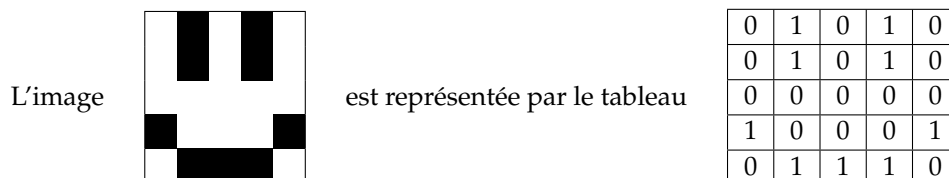
Dans cet exercice, on construit un type `Unir&Trouver` pour une partition d'un ensemble d'entiers $\llbracket 0; n - 1 \rrbracket$, avec la représentation sous forme d'arbre. On n'implémentera pas l'union par rang. Le type `Unir&Trouver` est donc un tableau d'entiers. Pour chaque élément, le tableau associe son parent dans l'arbre.

Dans un fichier `unirtrouver.ml`:

- Q1. Écrire un type `unirtrouver` pour cette implémentation.
- Q2. Écrire une fonction `init : int -> unirtrouver` qui prend en entrée le nombre d'éléments de la partition et renvoie une partition initiale où chaque élément est dans son propre sous-ensemble.
- Q3. Écrire une fonction `trouver : unirtrouver -> int -> int` qui renvoie le représentant d'un élément donné de la partition.
- Q4. Écrire une fonction `unir : unirtrouver -> int -> int -> unit` qui unit les sous-ensembles des deux éléments donnés en arguments.
- Q5. Écrire les tests pour ces fonctions et justifier leur pertinence.
- Q6. Si cela n'est pas déjà fait, implémenter la compression de chemin dans `trouver`.

Exercice 2. Coloration connexe Cet exercice est à réaliser dans un fichier `image.ml`.

Dans cet exercice, on souhaite colorier une image initialement en noir et blanc. Une image est représentée par un tableau en 2D dont les cases (pixels) contiennent un entier représentant une couleur. On suppose que, dans une image en noir et blanc, la valeur 0 correspond au blanc et le 1 au noir.



La coloration va consister à attribuer une couleur distincte à chaque composante connexe de pixels, c'est-à-dire les groupes de pixels adjacents (sans les diagonales) de même couleur. Sur l'image ci-dessus, on pourra par exemple avoir la coloration suivante, en utilisant les valeurs au-dessus de 1.



- Q1. Définir un type `image` pour représenter une image 2D sous forme de tableau 2D d'entiers.
- Q2. Créer une variable `smiley` qui représente le dessin en noir et blanc ci-dessus.
- Q3. Écrire une fonction `afficher_image_nb : image -> unit` pour afficher dans la console une image en noir et blanc en utilisant '.' pour les cases blanches et '#' pour les cases noires (ou toute autre couleur si l'image n'est pas en noir et blanc). On obtiendrait par exemple l'affichage suivant pour `smiley`.

```

. # . # .
. # . # .
. . . . .
# . . . #
. # # # .

```

On veut maintenant identifier les composantes connexes, c'est-à-dire les cases de même couleur qui sont connectées par un de leur côté commun (on ne considère pas les diagonales). On souhaite utiliser la structure `unirtrouver` précédente pour placer dans la même classe d'équivalence les cases de même couleur et connexes. Mais la structure utilise des entiers pour représenter ces éléments. Il nous faut donc une fonction `identifiant : int -> int -> int -> int` telle que `identifiant n i j` renvoie, pour une taille n de l'image, l'identifiant unique de la case (i, j) (la fonction doit être injective).

- Q4.** Proposer une implémentation de la fonction `identifiant : int -> int -> int -> int` et justifier brièvement son injectivité.
- Q5.** Écrire une fonction `composantes : image -> unirtrouver` qui retourne une partition correspondant aux composantes connexes de l'image.

On veut maintenant attribuer une couleur unique à chaque composante. Les couleurs sont numérotées dans $\mathbb{N}$. Pour colorer l'image, on va parcourir tous les éléments de la partition et pour chaque représentant rencontré lui attribuer une nouvelle couleur (= valeur) grâce à un compteur des couleurs déjà attribuées. Une fois la couleur attribuée à chaque représentant sur l'image, on peut attribuer leur couleur aux autres cases en consultant la couleur de leur représentant respectif.

- Q6.** Quel test effectuer sur la structure `unirtrouver` pour savoir que le i -ième élément est un représentant ?
- Q7.** Écrire une fonction `recolore : image -> unirtrouver -> image` qui retourne une image en couleur correspondant à l'image d'entrée où chaque composante connexe identifiée dans la partition `unirtrouver` a été coloriée avec une couleur qui lui est propre.
- Q8.** Écrire une fonction `afficher_image : image -> unit` qui affiche dans la console l'image « en couleur », on pourra utiliser les symboles ASCII à partir de la lettre 'a' pour chaque couleur (modulo 128). La coloration de l'image précédente sera alors :

```

a b a c a
a b a c a
a a a a a
d a a a e
f g g g h

```

Indication: Utiliser `Char.chr : int -> char` pour obtenir le caractère associé à un code ASCII et `Char.code 'a'` pour obtenir le code ASCII du caractère a.

II Implémentation par arbre en C

Dans cette partie, on utilise une implémentation par arbre en C (à récupérer sur CdP) pour implémenter un générateur de labyrinthe parfait. Vérifier tout d'abord la bonne compilation du code fourni avec le Makefile. Pour le moment le labyrinthe n'est pas affiché, on s'attend au résultat suivant en exécutant les commandes de compilation depuis le dossier où sont les sources :

```
$ make
gcc -c unirtrouver.c
gcc unirtrouver.o unirtrouver_labyrinthe.c -o unirtrouver_labyrinthe
$ ./unirtrouver_labyrinthe 10
Construction d'un labyrinthe de taille 10 x 10 ...
```

Pour l'instant, le programme n'affiche qu'un message avant de terminer. Après le message de construction on voudra afficher le labyrinthe créé.

Exercice 3. Fonctionnement par pointeurs

Le fichier `unirtrouver.c` contient l'ensemble des fonctions permettant de manipuler une structure `Unir&Trouver` (son en-tête associé contient les déclarations de ces fonctions).

Cette structure est différente de celle vue en cours. Les éléments désignent une structure avec la valeur de cet élément, le rang du sous-arbre de cet élément (comme dans le cours) et un pointeur vers l'élément parent (qui remplace l'indice du parent tel que défini dans le cours). Ainsi une racine a comme parent un pointeur sur lui-même.

La structure partition est alors un tableau d'éléments, c'est-à-dire un pointeur sur `element_t`.

Avec cette implémentation, il n'est pas nécessaire d'avoir tout le tableau des éléments pour effectuer une union ou retrouver une racine puisque l'on peut suivre les pointeurs.

- Q1.** Quelle est la structure de données obtenue en appelant `init_partition(3)` ? Représenter la structure en détaillant les pointeurs et les structures `element_t`. Expliquer comment est stocké le rang des éléments de la structure `Unir&Trouver`.
- Q2.** À quoi sert la fonction `free_partition` ?
- Q3.** Écrire une fonction `trouver_bis` sans récursivité. Est-il possible de conserver la compression de chemin avec cette version itérative ?
- Q4.** Ajouter les assertions nécessaires pour protéger les fonctions.

Exercice 4. Labyrinthe ★

Dans cet exercice on complètera le fichier `unirtrouver_labyrinthe.c`.

Le but de cet exercice est de se servir de la structure `Unir&Trouver` pour construire un labyrinthe *parfait*, c'est-à-dire un labyrinthe où il existe exactement un chemin pour lier deux cases. On représente un labyrinthe par un tableau en 2D tel que chaque case précise si elle a un mur à l'Est et un au Sud. La case du tableau contient un entier et on utilise la représentation binaire pour préciser quels murs sont présents. On a donc quatre valeurs possibles. Le bit de poids le plus faible précise s'il y a ou non un mur à l'Est. Le bit à sa gauche précise s'il y a ou non un mur au Sud. Le tableau ci-dessous récapitule, pour chaque valeur décimale, chaque code binaire représentant cette valeur décimale, quelle est la signification en terme de mur présent ou non dans la case.

Binaire	Décimal	Signification	Constante	Rendu visuel
0 0	0	Aucun mur	AUCUN_MUR	
0 1	1	Mur à l'Est	MUR_EST	
1 0	2	Mur au Sud	MUR_SUD	—
1 1	3	Murs sur les deux côtés	DEUX_MURS	—

Remarque (HP). Pour utiliser la représentation binaire, on dispose en C d'opérateurs binaires, qui appliquent une opération bit à bit sur les opérandes.

Le OU bit à bit , $a \mid b$, applique le OU logique sur chaque bit de a et b .

Ex: $0110 \mid 1100 = 1110$.

Le ET bit à bit , $a \& b$, applique le ET logique sur chaque bit de a et b .

Ex: $0110 \& 1100 = 0100$.

Le XOR bit à bit , $a \wedge b$, applique le OU exclusif logique sur chaque bit de a et b .

Ex: $0110 \wedge 1100 = 1010$.

On peut donc vérifier si une valeur v correspond à une case avec un mur à l'Est avec $v \& \text{MUR_EST} \neq 0$ (ou $v \& \text{MUR_EST} = \text{MUR_EST}$). L'instruction renverra True qu'il y ait juste un mur à l'Est ou deux murs, à l'Est et au Sud.

Le tableau 2D peut être représenté par un tableau à une dimension : pour accéder à la case (i, j) d'un tableau 2D de taille $n \times n$, on accède à la case $i \times n + j$ dans le tableau à une dimension. *Il est recommandé d'écrire une fonction intermédiaire pour ce calcul, afin d'éviter des erreurs de copier-coller et pour mettre des assertions au début de cette fonction.*

- Q1. Écrire une fonction `int *init_labyrinthe(unsigned int taille)` qui renvoie un pointeur vers un tableau de taille `taille × taille`. Chaque case du tableau correspond à une case du labyrinthe, initialement avec un mur à l'est et un au sud.
- Q2. Écrire une fonction `void afficher_labyrinthe(int *labyrinthe, unsigned int taille)` pour afficher le labyrinthe dans la console. On souhaite un rendu tel que ci-dessous. En particulier, on suppose que le labyrinthe est entièrement entouré de murs. Les côtés Est et Sud sont gérés par les cases elles-même, mais la fonction d'affichage doit tout de même afficher la bordure Nord et Ouest.

```

- - - - -
|_ _ _ | _ _ | | | |
| _|_ _| _| _| _|
|_ | _|_ _| _| _|
|_ | _ _|_ _|_ _|
| _ _|_ _| _| _|
| _| _| _| _| _|
| _ _| _| _ _| _|
| _| _| _| _| _|
| _| _| _| _| _|
| _ _ _ _|_ _|_ _|

```

Ci-dessus, le tableau est de taille 10×10 et la première case, en haut à gauche, a un mur Sud mais pas de mur Est. La case à sa droite n'a aucun mur Sud ni Est.

Indication 1: Utiliser l'opérateur `&` pour vérifier si une case a un certain type de mur.

Indication 2: On décompose l'affichage ainsi:

- Une boucle affiche le mur Nord.
- Une boucle affiche chaque ligne, en commençant systématiquement par afficher un mur Ouest. On suppose le labyrinthe bien construit donc les cases à l'Est doivent toutes avoir un mur Est. De même toutes les cases au Sud doivent avoir un mur Sud. On ne vérifiera pas cette information.

Pour construire le labyrinthe parfait on utilise une structure Unir&Trouver où chaque élément est une case du tableau. Deux cases appartiennent au même sous-ensemble de la partition s'il existe un chemin entre les deux.

On ne supprime donc un mur entre deux cases que si elles appartiennent à des classes différentes, classes que l'on unit après avoir supprimé le mur.

Pour construire le labyrinthe, on considère chaque mur Est et Sud de chaque case (sauf celle à l'Est et au Sud du labyrinthe). Ce mur peut être représenté par la paire des cases adjacentes à ce mur. L'algorithme prend chaque paire de cases adjacentes dans un ordre aléatoire et on décide de supprimer ou non le mur selon la classe d'équivalence de ces deux cases. Ainsi, lorsque le parcours des paires est terminé on sait qu'il existe exactement un chemin entre chaque case dans le labyrinthe.

Exemple. On prend la grille 2×2 suivante dont on numérote les cases. Les murs définis pour cette grille sont alors : $\{(0, 1), (0, 2), (1, 3), (2, 3)\}$. On va mélanger cette liste de mur pour construire un labyrinthe parfait aléatoire :

0	1
2	3

→ Suppressions dans l'ordre $[(2, 3), (0, 2), (0, 1), (1, 3)]$ →

0	1
2	3

La suppression du dernier mur, $(1, 3)$ n'a aucun effet car à ce stade les cases 1 et 3 sont déjà connectées par un chemin et déjà dans la même classe dans la structure Unir&Trouver.

Q3. Définir une structure `mur` contenant deux champs correspondants aux indices des deux cases adjacentes à ce mur. On se servira de cette structure pour manipuler les paires de cases.

Q4. Pour un tableau de taille n , combien de murs doit-on considérer ?

Q5. Pour parcourir les murs dans un ordre aléatoire, on crée la liste des murs que l'on va ensuite mélanger. Écrire une fonction `mur *liste_murs(unsigned int taille)` qui renvoie l'ensemble des murs de cases adjacentes existants dans un tableau de taille $taille \times taille$.

Indication: Inutile de créer la paire de cases $(i, j), (i + 1, j)$ et la paire $(i + 1, j), (i, j)$. Pour chaque case on peut donc se contenter de créer deux paires, celle avec la case au Sud et celle avec la case à l'Est: $(i, j), (i + 1, j)$ et $(i, j), (i, j + 1)$.

Indication: Attention aux cases des bords Est et Sud !

Q6. Écrire une fonction `void permuter_murs(struct mur* liste_murs, int premier, int second)` qui permute dans la liste donnée en entrée les murs d'indices `premier` et `second`.

Le *mélange de Knuth* est un algorithme de mélange des éléments d'un tableau.

Algorithme 1 : Mélange de Knuth

```

1 pour i = 0 à |tableau| - 1 faire
2   j ← Entier aléatoire entre 0 et i (inclus)
3   Échanger les cases d'indices i et j dans le tableau
```

Q7. Écrire une fonction `void melanger(struct mur *liste_murs, unsigned int taille)` qui applique le mélange de Knuth sur la liste de murs. Pour cela on appellera `rand()` (voir Annexes).

Indication: On utilisera un modulo pour restreindre l'intervalle de valeurs aléatoire.

Q8. Écrire une fonction `void construire(int *labyrinthe, unsigned int taille)` qui reçoit en entrée une grille `labyrinthe` tel que ses cases ont toutes un mur Sud et Est, et qui la modifie pour en faire un labyrinthe parfait en appliquant les étapes suivantes :

- Création de la liste des murs
- Mélange de Knuth sur cette liste
- Création d'une partition pour les murs
- Parcours de la liste mélangée : pour chaque mur, si les classes des deux cases sont différentes alors le mur est supprimé de `labyrinthe` et les deux cases fusionnent leurs classes.

Pour aller plus loin Implémenter un parcours en profondeur et une fonction d’affichage pour montrer le chemin de la première case (en haut à gauche) à la dernière (en bas à droite).

```

- - - - -
|_   _ | | |_ | . .
| _|_ | | | | _| . .
|_ | _ | _| _| | _| . .
|_ | _ | _| _| _| .
| _ _|_ | | _| _| . . . .
| | _ _|_ _|_ | . . . .
| | _ | _| _| _| . . . .
| _ _| _| _ . . . . .
| _| | | | _| | | . . . .
|_ - - - _|_ _|_ _|_ | . . . .

```

Annexes

Pour utiliser `rand()` il faut importer `stdlib.h`. Ce header définit une constante `RAND_MAX` (constante = *vous ne pouvez pas la modifier*). Pour obtenir un entier aléatoire entre *a* et *b* (inclus) on applique donc une division euclidienne :

```
1 int nb = a + (rand() % (b - a + 1))
```

Sans plus de précaution, la suite d’appels à `rand()` donnera la même séquence de nombres à chaque exécution du programme (car il utilise un générateur pseudo-aléatoire). Pour éviter cela on initialise l’aléatoire avec une nouvelle *graine* à chaque exécution. Cette graine est choisie comme étant l’instant présent selon l’horloge de l’ordinateur. L’initialisation de cette graine est déjà faite dans le code source fourni, au début de la fonction `main` :

```
1 srand(time(NULL));
```